

Explicit concept expressions

Thomas Köppe <tkoeppe@google.com>Hubert S.K. Tong <hubert.reinterpretcast@gmail.com>

Abstract

We propose to remove from the working paper the fact that plain concept names can be boolean prvalue expressions and instead introduce a mildly more verbose “*simple-requires-expression*”. Instead of `bool b = Sortable<MyType>;` we propose `bool b = requires Sortable<MyType>;` This change removes unfortunate corner cases and paves the way for future extensions.

Contents

1. [Before/After](#)
2. [Problem and motivation](#)
3. [Proposal](#)
4. [Alternatives](#)
5. [Impact on the past and future](#)
6. [Proposed wording](#)
7. [Acknowledgements](#)

Before/After

Before the proposal

```
template <typename T>
concept Foo = /* ... */;

template <typename T>
void f(T) {
```

With the proposal

```
template <typename T>
concept Foo = /* ... */;

template <typename T>
void f(T) {
```

Before the proposal	With the proposal
<pre>static_assert(Foo<T>); if constexpr (Foo<T>) { /* ... */ }</pre>	<pre>static_assert(requires Foo<T>); if constexpr (requires Foo<T>) { /* ... */ }</pre>

Problem and motivation

Consider a simple type concept:

```
template <typename T, typename U = T> concept Foo = /* ... */;
```

For a given type, say `int`, the current working paper makes `Foo<int>` a boolean prvalue. This creates a curious corner case if we consider future syntax extensions for abbreviated function templates. Consider the following code:

```
template <typename T> bool x(Foo<T>);
```

There are two plausible meanings this code could have (assuming further work in the direction of [P0745R0](#)):

```
template <typename T>                                bool x =
Foo<T>;    // #1 (variable)

template <typename T, typename U> requires Foo<T, U> bool x(U u);
// #2 (function)
```

This is kind of an inverse of the “most vexing parse”. In the status quo of the working paper, the meaning is that of case #1 (a variable declaration); whereas at least some have expressed that the preferable meaning would be that of case #2 (a function declaration). This latter case is the one that is consistent with dropping constraints: `template <typename T> bool x(T)` is already a function declaration in the status quo.

We can avoid this problem if we make it so that plain concept names do not form prvalue expressions, thus spuriously occupying a “privileged” syntax. If we retain the status quo and later discover that this problem is serious, it will be a breaking change to remove or change the behaviour. On the other hand, if we remove the behaviour now and later discover that we actually do need it, we can easily add it back in without breaking code written to a level of C++ specified by an International Standard.

Proposal

We propose that an *id-expression* ([*expr.prim.id*]) that denotes the specialization of a concept no longer results in a prvalue except in specific contexts where we can expect normalization to consider it to be an atomic constraint. To make up for the lost functionality, we introduce a new kind of expression: a “*simple-requires-expression*”, which turns a specialization of a concept into the boolean value indicating its constraint satisfaction.

Alternatives

Instead of making concepts not be expressions, we could make them expressions precisely when they are “named fully” or “fully specialized”, but treat them as type placeholders when they are only “partially bound”. A hypothetical short function template syntax might then fit in like this:

```
template <typename A, typename B> concept Foo = /* ... */;

bool x(Foo<int, char>); // variable, equiv. to bool x = Foo<int,
char>;
bool y(Foo<char>);      // function, equiv. to template
<typename T> requires Foo<char, T> bool y(T)
```

In this approach, it is necessary to know the declaration of `Foo` in order to know whether `bool y(Foo<char>)` declares a variable or a function, with the usual concerns about maintenance, default arguments, etc.

This approach presupposes a detail of a future, not-yet-made design. It is a plausible approach, but at the same time this direction is compatible with our main proposal: we can first remove the valueness of plain concept names, and later bring it back with the semantics described here.

The proposed new *simple-requires-expression* is not strictly required, since a boolean value can also be obtained from an (ordinary) *requires-expression* such as “`requires { requires Foo<T>; }`”. We feel that such an expression would be somewhat unwieldy in contexts such as those of `constexpr` if and static assertion, and that the new *simple-requires-expression* will fit in more naturally.

Impact on the past and future

There is no impact on the Standard, since the proposal modifies a feature that has not yet been standardized.

Removing concept *id-expressions* now means that we do not standardize a feature that we may later regret and cannot change without a break. On the other hand, adding the removed feature back in later, if and when we do need it, is straightforward.

Proposed wording

[to be fleshed out]

requires-expression:

`requires` *requirement-parameter-list*_{opt} *requirement-body*

simple-requires-expression:

`requires` *nested-name-specifier*_{opt} *simple-template-id*

An ~~*id-expression*~~ *simple-requires-expression* whose *simple-template-id* denotes the specialization of a concept [temp.concept] results in a prvalue of type `bool`. The expression is `true` if the concept’s normalized *constraint-expression* [temp.constr.decl] is satisfied [temp.constr.constr] by the specified template arguments and `false` otherwise.

```
template<typename T> concept C = true;
static_assert(requires C<int>); // OK
```

[Note: A concept’s constraints are also considered when using a template name [temp.names] and during overload resolution [over], and they are compared during the the partial ordering of constraints [temp.constr.order]. —end note]

Acknowledgements

Many thanks Andrew Sutton for valuable discussion.